

**LABORATORY ACTIVITY NO. 2.3.8**

Using Scripting Tools

Members:

**NACIONALES, CHEVELLE S.
FRANCISCO, SEAN GABRIEL P.**

BSIT – 102i

This lab begins by teaching the creation of a simple batch file to automate the generation of a network configuration report. It then moves to Visual Basic Scripting, demonstrating how to create a basic loop program and showcasing how VBScripts can be executed by double-clicking them or via the *cscript* command in the command prompt. The VBScript exercise progresses to include more complex features like generating random numbers and using conditional statements with message boxes. Finally, the lab introduces Windows PowerShell, illustrating how to use cmdlets to retrieve system information and filter/sort results. It also covers creating and saving PowerShell scripts in the PowerShell ISE, and addresses the security measure of execution policies, demonstrating how to allow unsigned scripts to run.

I. OBJECTIVES

Upon completion of this laboratory exercise, students are expected to and meet the following intended learning outcomes:

Activity Objectives	Intended Learning Outcomes		
	ILO1	ILO2	ILO3
1) create and execute basic batch scripts to automate command-line tasks, understanding file saving procedures and command prompt interactions.	•	•	•
2) Develop and test Visual Basic Scripts (VBScripts) to perform operations such as loops and conditional logic, and understand how to run these scripts from both File Explorer and the command.	•	•	
3) Utilize Windows PowerShell to run cmdlets for system information retrieval and manipulate output using pipes and formatting. Additionally, learn to create and execute PowerShell scripts, including managing the execution policy for security.	•	•	



Where:

- ILO1 – Exhibit thorough demonstration of the concept
- ILO2 – Being able to perform the activity independently
- ILO3 – Design and evaluate emerging IT solutions

II. DISCUSSION

- **Programming Logic**

- Focuses on software development, programming languages, and the importance of clear program sequence, input, processing, and output.
- Illustrates program flow using flowcharts and explains the concept of pseudocode, emphasizing breaking down steps into simple instructions and using subroutines.

- **Features of Programming Code**

- Highlights common constructs across programming languages, including subroutines (user-defined), return statements, conditional statements (if, if-else), variables, functions (pre-defined), user interfaces (console, GUI, web), and comments.

- **Programming Languages**

- Discusses factors in choosing a language, such as program type, programmer experience, and target devices.
- Lists popular languages like Java, Python, JavaScript, C++, C#, PHP, Perl, and Ruby.
- Differentiates between **compiled languages** (transformed to executable binary, fast but platform-specific, e.g., C++, C#) and **interpreted languages** (run within an interpreter, slower but platform-independent, e.g., Java, JavaScript, Python).
- Covers **query languages** (for databases, e.g., SQL), **assembly language** (human-readable machine code), and **markup languages** (for document structure, e.g., HTML, XML).

- **Identifiers**

- Explains variables (values that can change) and constants (values that do not change) in programming.



- Introduces identifiers for multiple elements like arrays (fixed-size, same data type) and vectors (dynamic arrays).
- **Branches and Loops**
 - Describes control flow mechanisms: branches (execute code based on condition, e.g., If...Then) and loops (repeat code based on condition, e.g., For...Next, Do While).
- **Operators**
 - Details comparison operators (e.g., ==, !=, <, >) and logical operators (AND, OR, XOR).
- **Procedures, Functions, and Comments**
 - Explains that procedures and functions are used for blocks of code, with functions specifically returning a value.
 - Highlights the use of comments for documenting code, noting they are not executed by the compiler/interpreter.
- **Object-Oriented Programming (OOP)**
 - Introduces the concept of designing code to represent "things" (objects) that interact through defined methods.
 - Defines attributes (values/data types of an object), methods (what can be done to an object), and properties (alternative public access to fields).
- **Scripting Languages**
 - Describes scripting languages as interpreted, often used for OS automation, smaller than programs, and designed for specific tasks (e.g., Batch files, Windows PowerShell, VBScript, Bash).
- **Application Platforms and Delivery Methods**
 - Categorizes applications by platform (single-platform, cross-platform, web applications) and delivery methods (local installation, network installation, cloud-hosted).

III. MATERIALS

A Windows 11 installed computer system.



IV. PROCEDURE

For this activity, you will execute the provided step-by-step instructions. As you complete each step, capture a screenshot of the resulting output or completed action that directly demonstrates successful execution of that instruction. Paste each screenshot into the designated area within your lab document.

Important Considerations:

- **Accessing Notepad (Windows 11):**
 - Instead of "Press START, type notepad and then press ENTER", in Windows 11, you would click the **Start button** (usually centered on the taskbar), type notepad in the search bar that appears, and then press Enter.
- **Accessing Command Prompt (Windows 11):**
 - To open Command Prompt, instead of "Click Start, scroll down and expand Windows System, and then click Command Prompt", in Windows 11, click the **Start button**, type cmd in the search bar, and then click "Command Prompt" from the search results.
- **Accessing Windows Explorer (Windows 11):**
 - Instead of "Open Windows Explorer and navigate to your Documents folder", in Windows 11, you can simply click the **File Explorer icon** on the taskbar (looks like a folder) or press Windows key + E, then navigate to your Documents folder.
- **Accessing Windows PowerShell / PowerShell ISE (Windows 11):**
 - For "Click Start, type Windows PowerShell, right-click Windows PowerShell and then click Run as administrator" , and "Click Start, type Windows PowerShell, right-click Windows PowerShell ISE and then click Run as administrator", in Windows 11:
 - Click the **Start button**.
 - Type Windows PowerShell or Windows PowerShell ISE in the search bar.
 - Right-click the relevant application in the search results and select "Run as administrator."
 - **User Account Control (UAC) Prompts:** Be prepared to click "Yes" on any User Account Control (UAC) prompts that appear when running applications or making system changes as an administrator.



- **File Extensions:** Pay close attention to saving files with the correct extensions: .cmd for batch files , .vbs for Visual Basic Scripts , and .ps1 for PowerShell scripts. Incorrect extensions will prevent the scripts from running as intended.
- **Script Content Accuracy:** Type the script code exactly as provided in the instructions, including special characters and spacing, as syntax errors can prevent scripts from running.
- **PowerShell Execution Policy:** Understand that PowerShell has a default execution policy that blocks unsigned scripts for security. The lab specifically instructs you to change this policy to

RemoteSigned to allow your created script to run. It's crucial to be aware of the security implications when modifying execution policies on a production system.

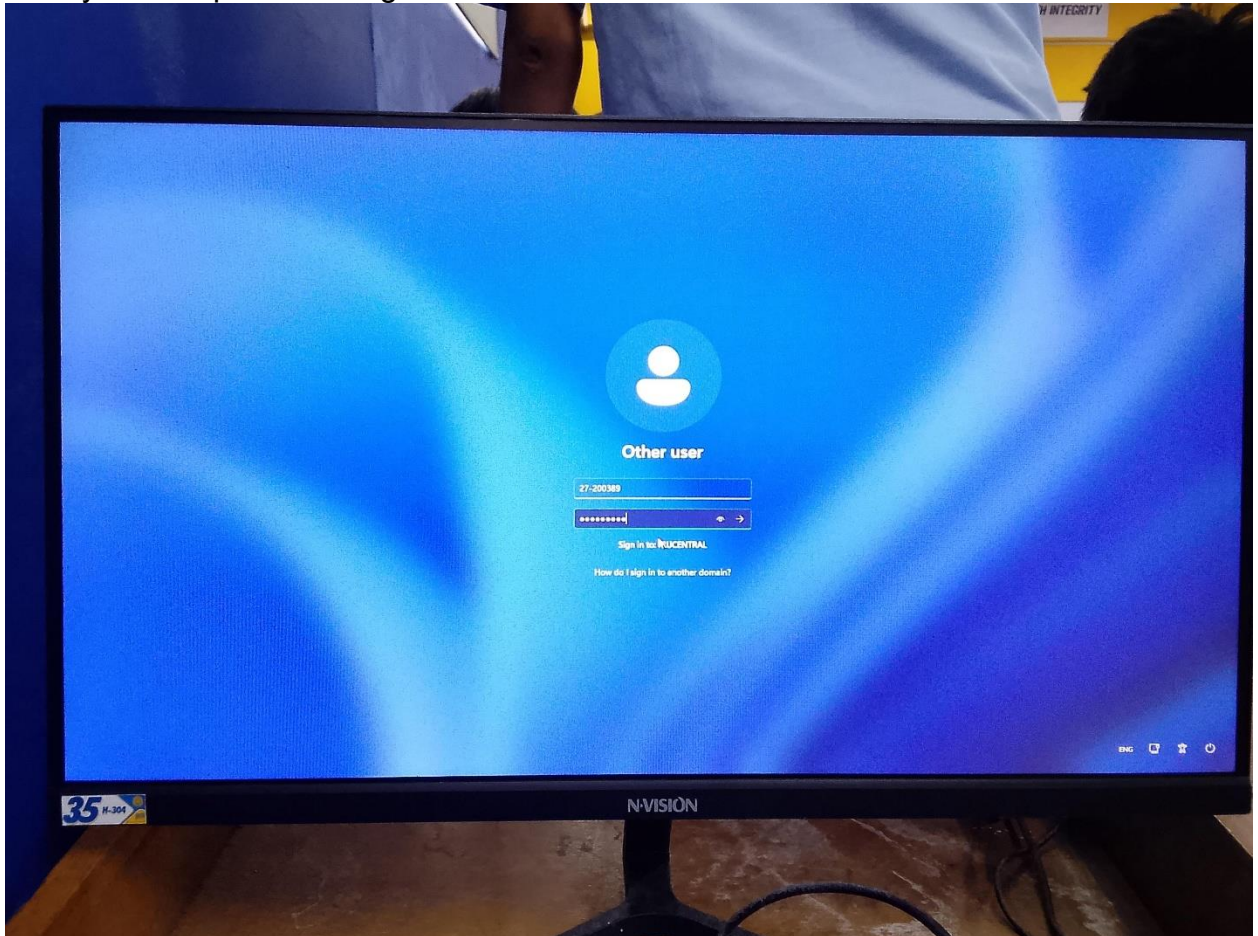
- **Piping Output:** The PowerShell exercises demonstrate the use of the pipe (|) operator to send the output of one command (cmdlet) as input to another, which is a fundamental concept in PowerShell for chaining commands.
- **Command Line vs. ISE:** The lab highlights the difference between running PowerShell cmdlets directly in the command line and creating and executing scripts within the PowerShell Integrated Scripting Environment (ISE), which offers features like color-coding and autocompletion to aid in development.

Steps:

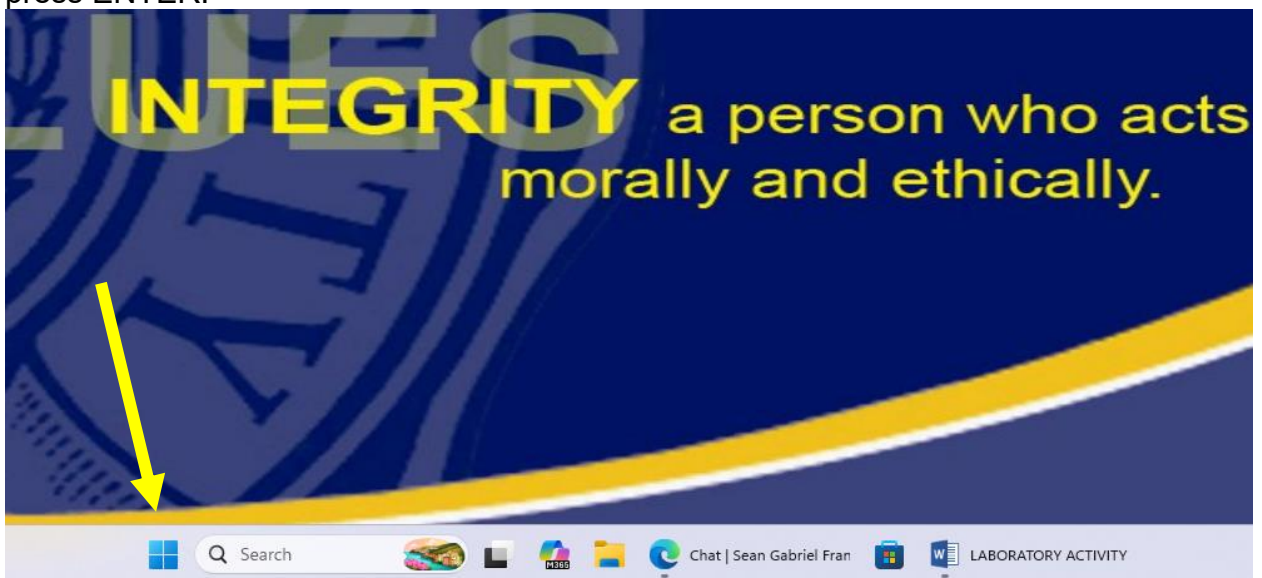
Part 1: Creating a Batch File

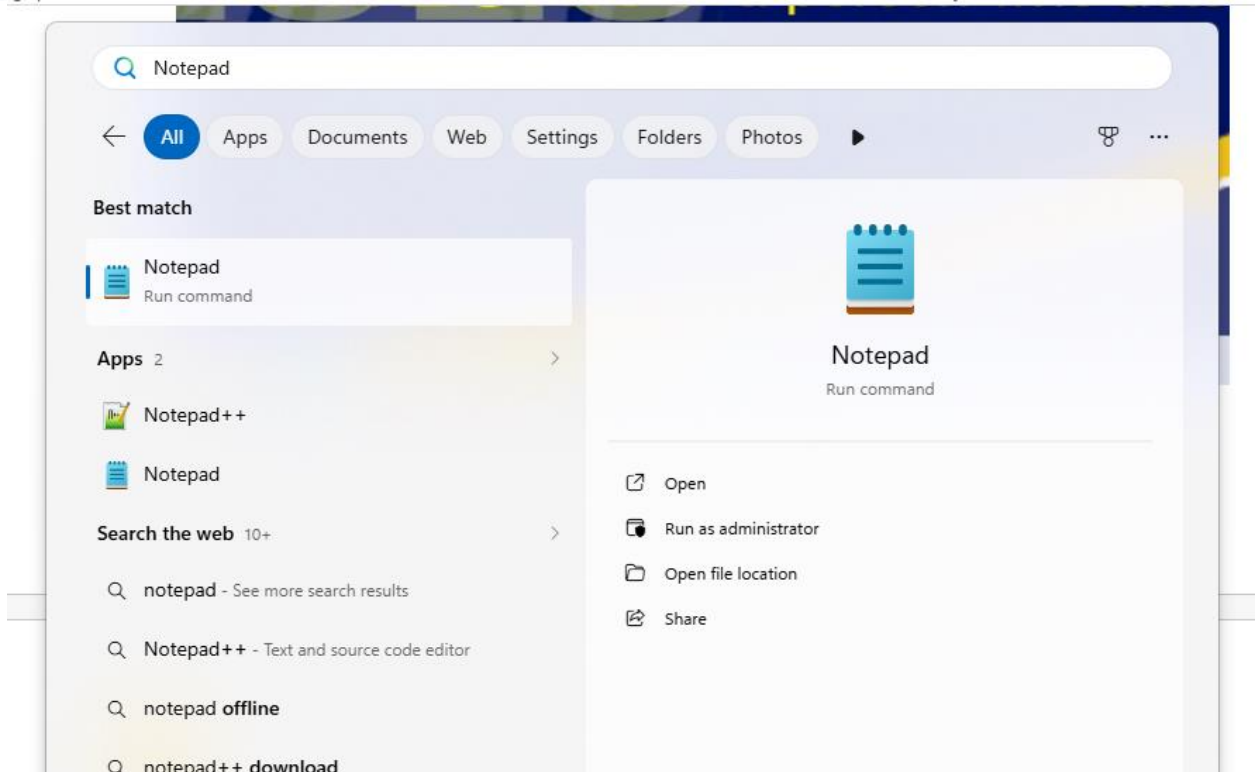


1. Start your computer and sign in.



2. Click the **Start button** (on the taskbar), type notepad in the search bar, and then press ENTER.





3. In the Untitled - Notepad window, type the following text:

```
@echo off
ipconfig /all > report.txt
echo Press a key to view report
pause > nul
notepad report.txt
```

○ **Note the features of the script:**

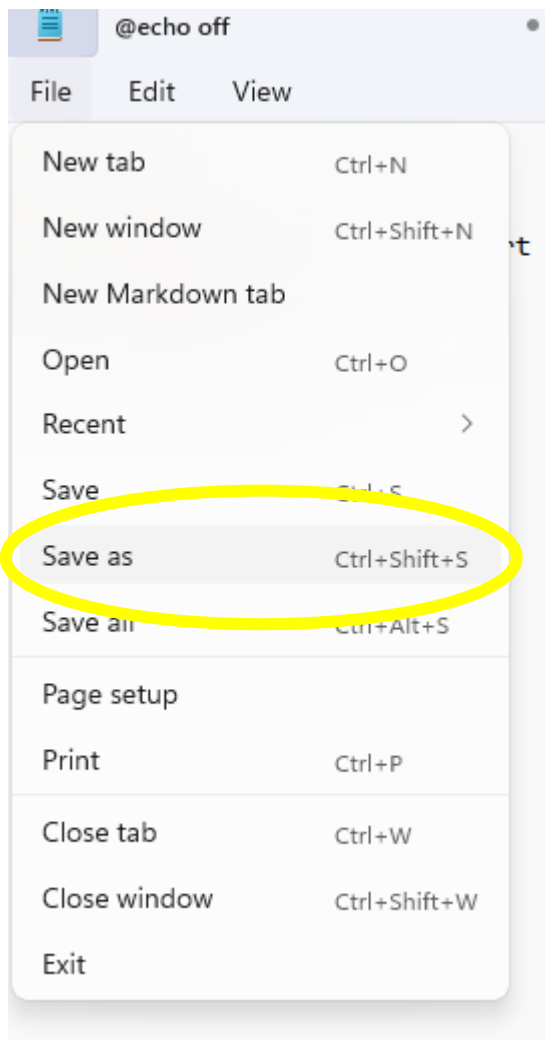
- @echo off - suspends output being written to the command prompt.
- ipconfig /all > report.txt - calls the system command to produce a network configuration summary (ipconfig) and redirects (or pipes) its output to a text file.
- The remainder of the script waits for the user to press a key and then opens the report file in Notepad.



```
@echo off
ipconfig /all > report.txt
echo Press a key to view report
pause > nul
notepad report.txt
```

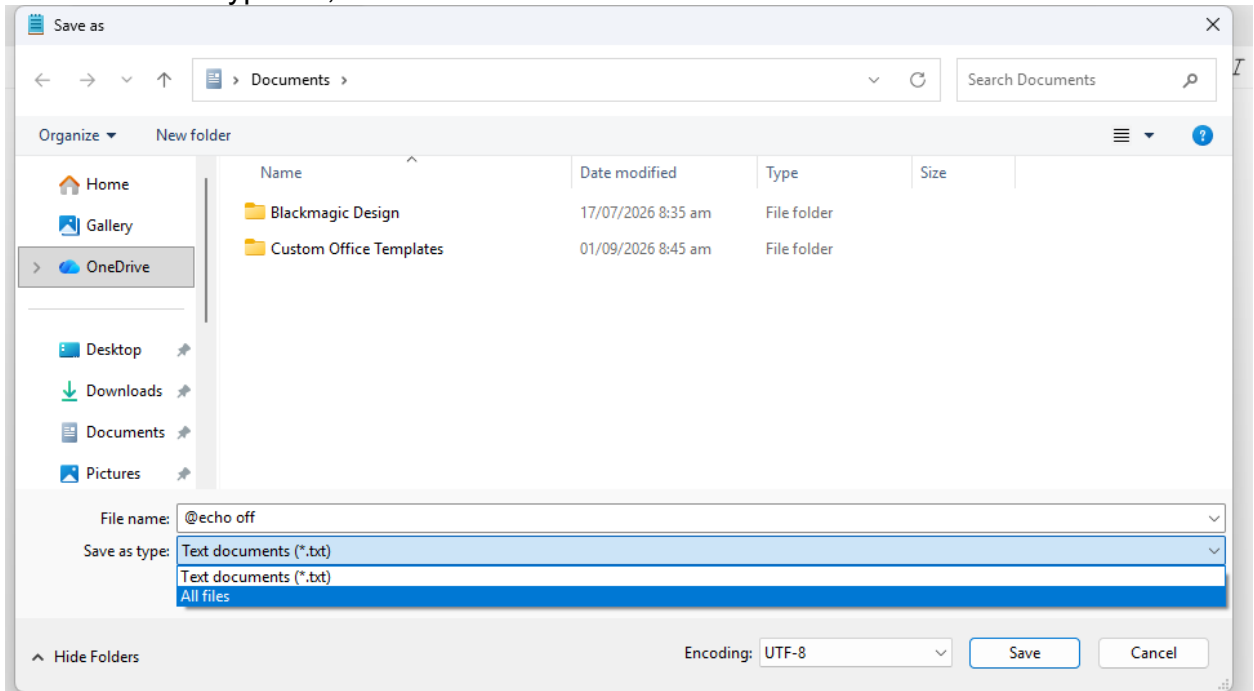


4. From the File menu, select **Save As**.

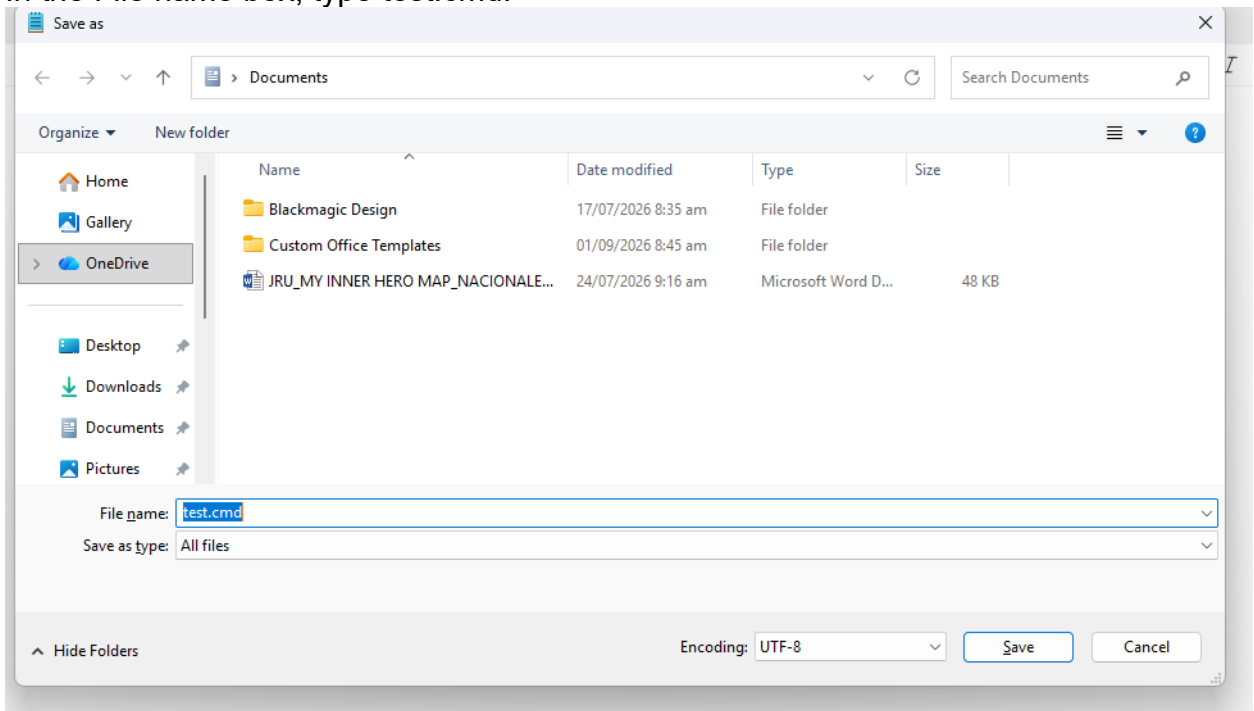




5. In the Save as type list, click **All Files**.

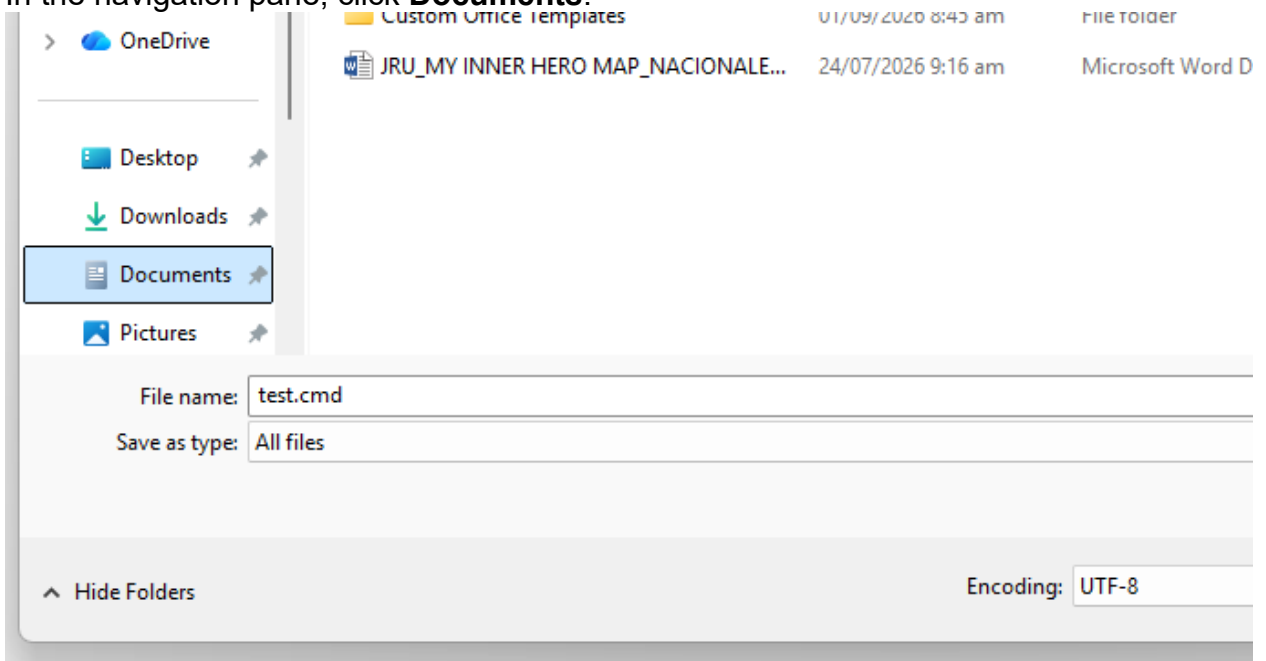


6. In the File name box, type test.cmd.

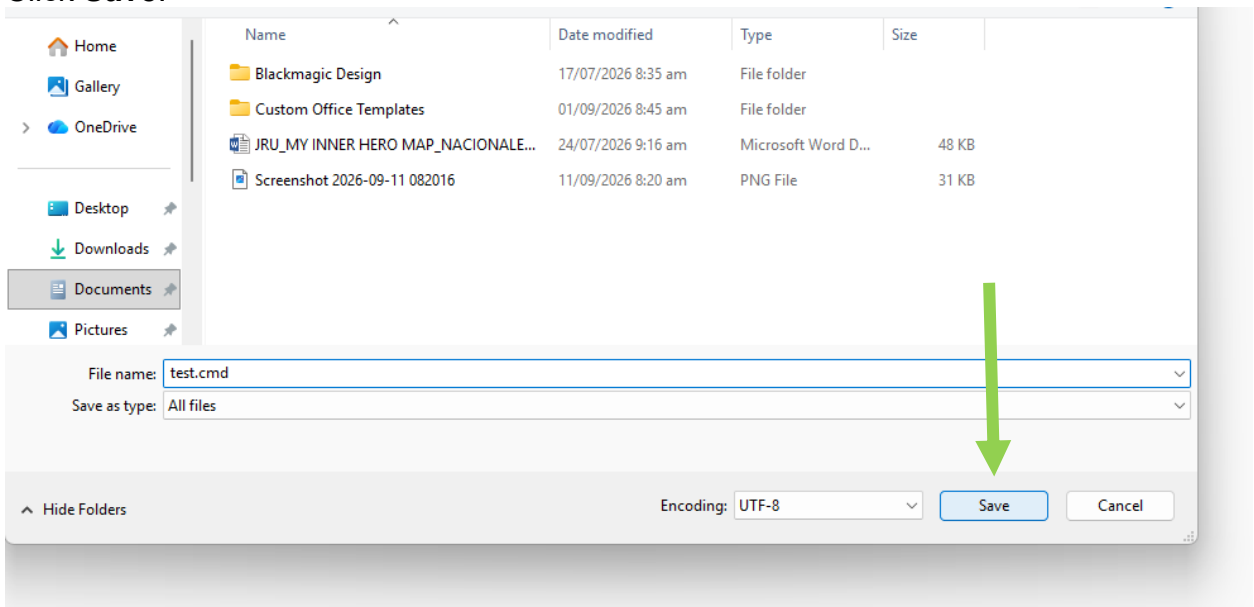




7. In the navigation pane, click **Documents**.



8. Click **Save**.

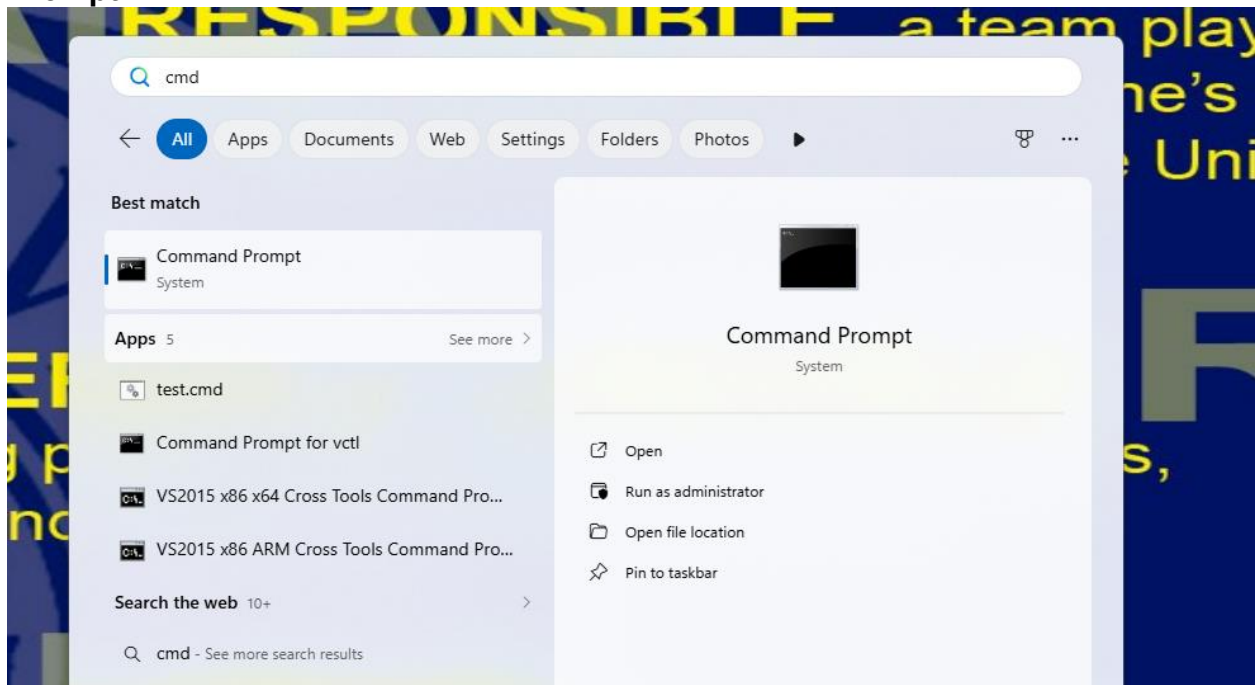




9. Close Notepad.



10. Click the **Start button**, type cmd in the search bar, and then click **Command Prompt** from the search results.





11. In the Command Prompt window, type `documents\test.cmd` and then press ENTER.

```
Command Prompt
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.

C:\Users\27-200389>documents\test.cmd
```

12. When prompted to press a key, press **SPACEBAR**.
- A Notepad window opens and displays the output from the `ipconfig.exe` command that you put in your batch file.

```
Command Prompt - document
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.

C:\Users\27-200389>documents\test.cmd
Press a key to view report
|
```



ITC C103 – IT Fundamentals

```
test.cmd  report.txt
File Edit View H1 H2 H3 B I S E A
Windows IP Configuration

Host Name . . . . . : H304-35
Primary Dns Suffix . . . . . : icucentral
Node Type . . . . . : Hybrid
IP Routing Enabled. . . . . : No
WINS Proxy Enabled. . . . . : No
DNS Suffix Search List. . . . . : icucentral

Ethernet adapter Ethernet 2:

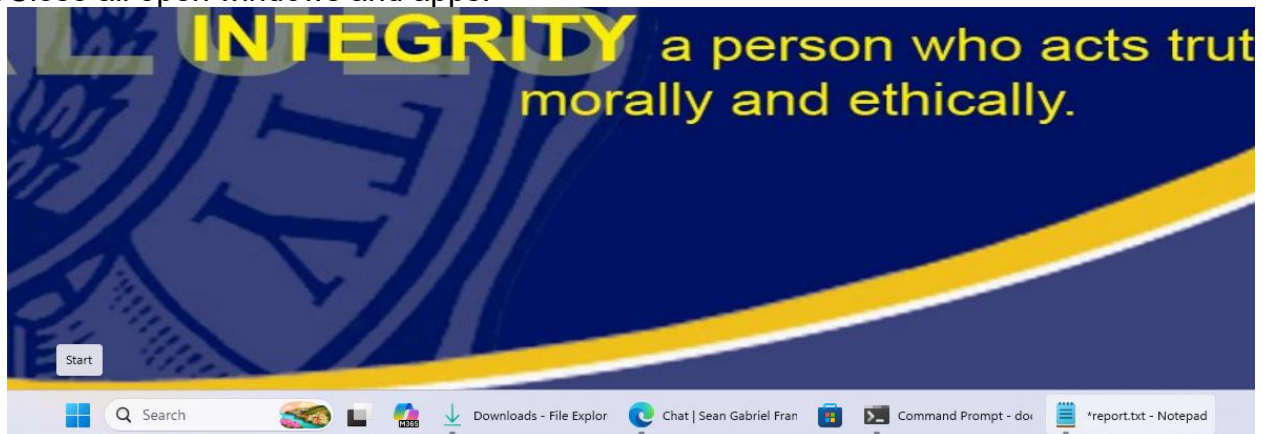
Connection-specific DNS Suffix . : 
Description . . . . . : Realtek PCIe GBE Family Controller #2
Physical Address. . . . . : CC-28-AA-0B-F7-A7
DHCP Enabled. . . . . : No
Autoconfiguration Enabled . . . : Yes
IPv4 Address. . . . . : 168.212.3.45(Preferred)
Subnet Mask . . . . . : 255.255.0.0
Default Gateway . . . . . : 168.212.1.4
DNS Servers . . . . . : 168.212.1.7
                        168.212.1.4
NetBIOS over Tcpip. . . . . : Enabled

Ethernet adapter Ethernet 3:

Connection-specific DNS Suffix . : 
Description . . . . . : VirtualBox Host-Only Ethernet Adapter
Physical Address. . . . . : 0A-00-27-00-00-11
DHCP Enabled. . . . . : No
Autoconfiguration Enabled . . . : Yes
Link-local IPv6 Address . . . . : fe80::42df:1982:8338:29d3%17(Preferred)
IPv4 Address. . . . . : 192.168.56.1(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . :

Ln 1, Col 1 3,534 characters Plain text 100% Windows (CRLF)
```

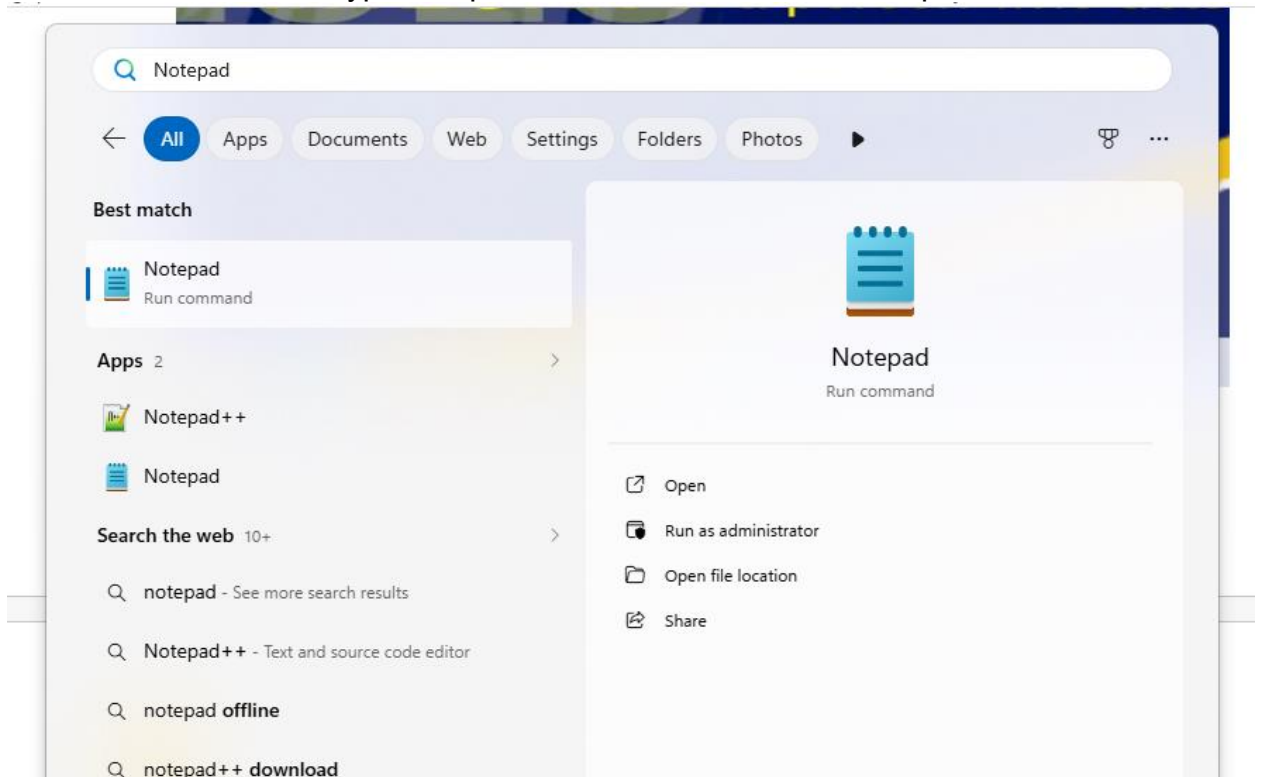
13. Close all open windows and apps.





Part 2: Creating a Visual Basic Script

1. Click the **Start button**, type notepad in the search bar, and then press ENTER.





2. In the Untitled - Notepad window, type the following text:

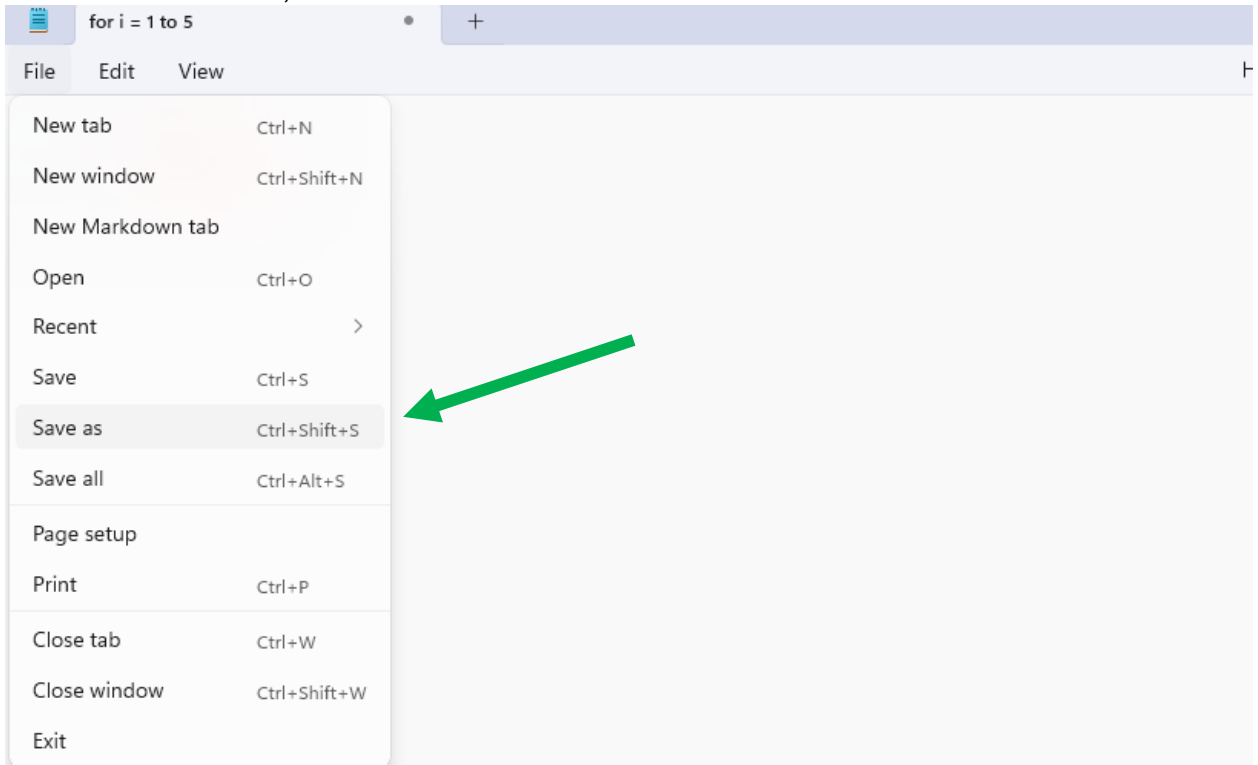
```
for i=1 to 5
    wscript.echo i
next
```

- This program creates a simple loop and uses the echo statement to show the current value of the variable *i* as the loop executes.

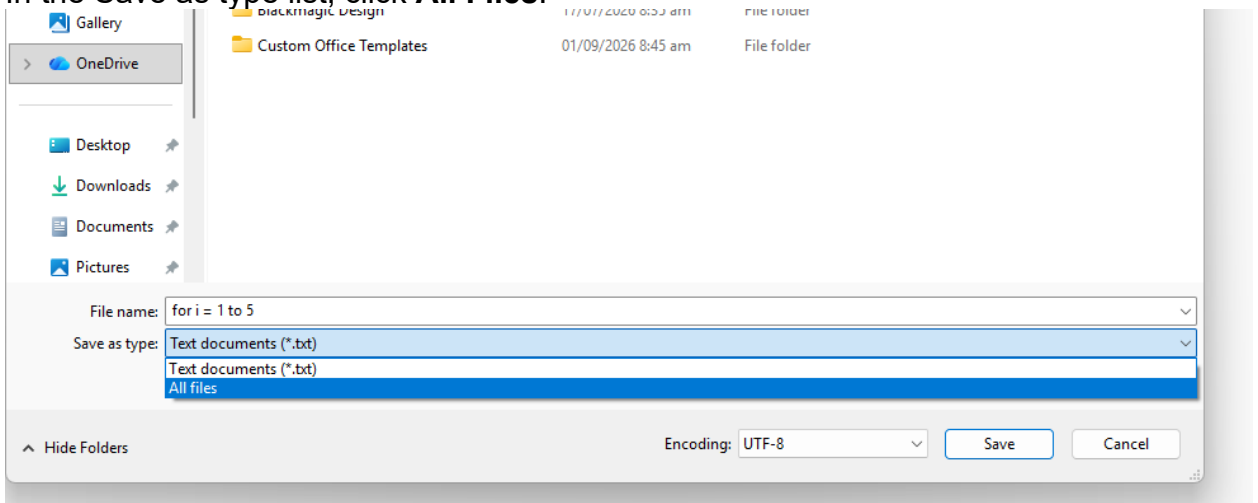
```
for i = 1 to 5
    wscript.echo i
next
```



3. From the File menu, select **Save As**.

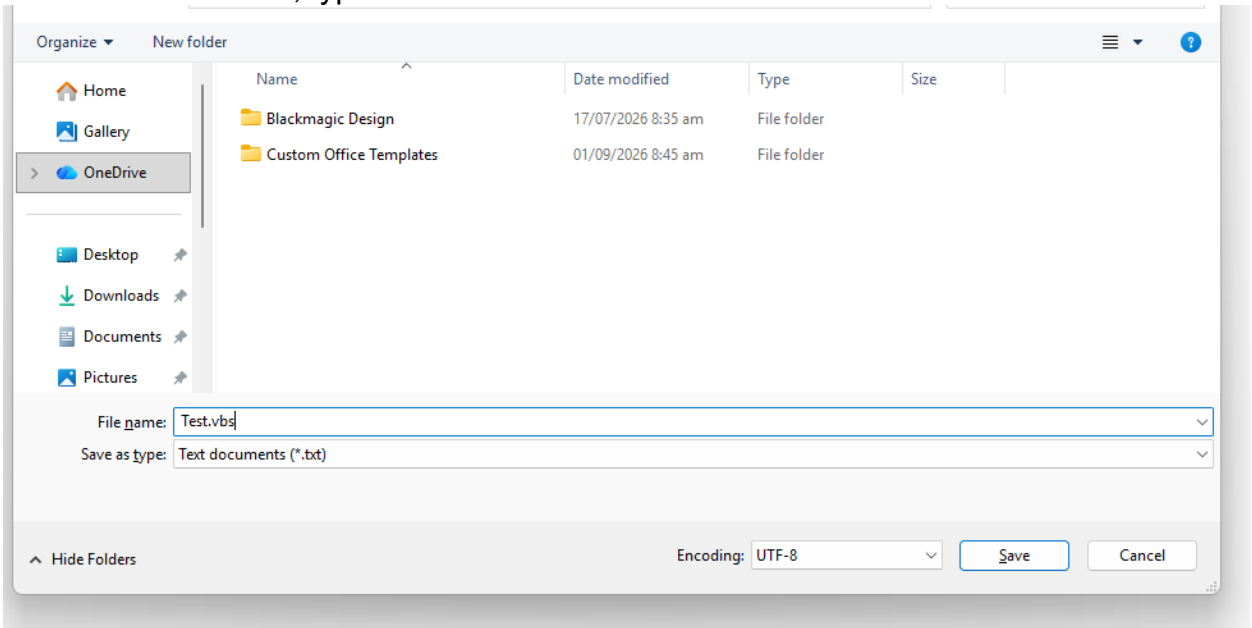


4. In the Save as type list, click **All Files**.

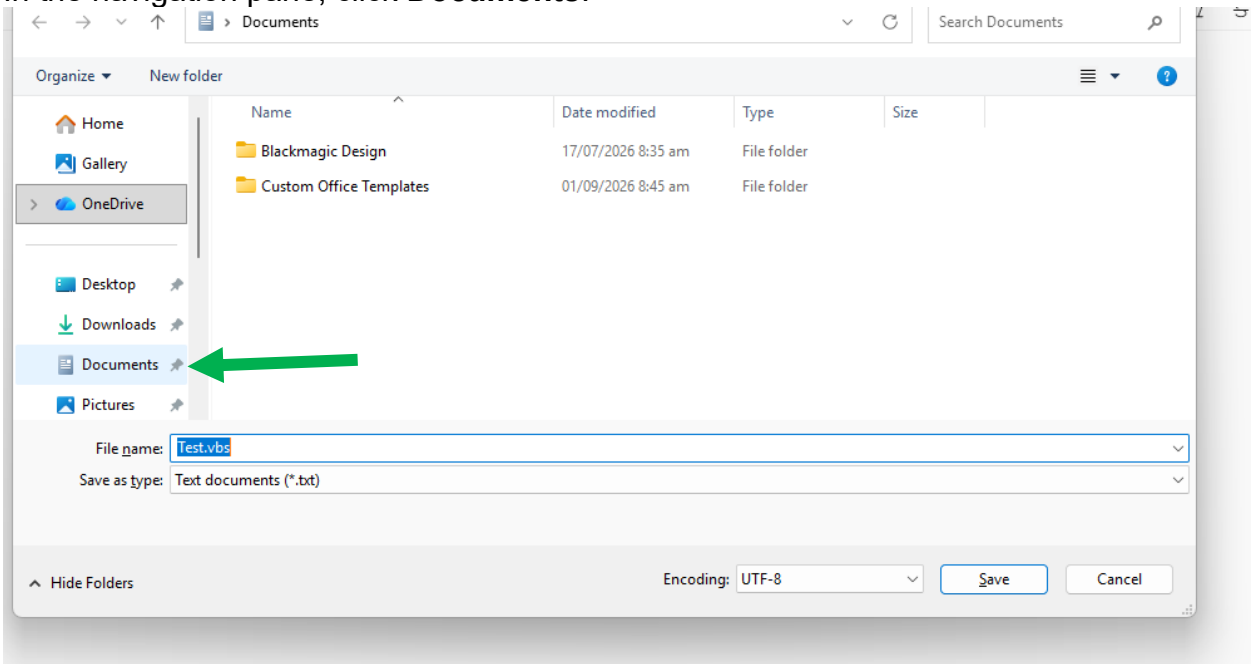




5. In the File name box, type *Test.vbs*.

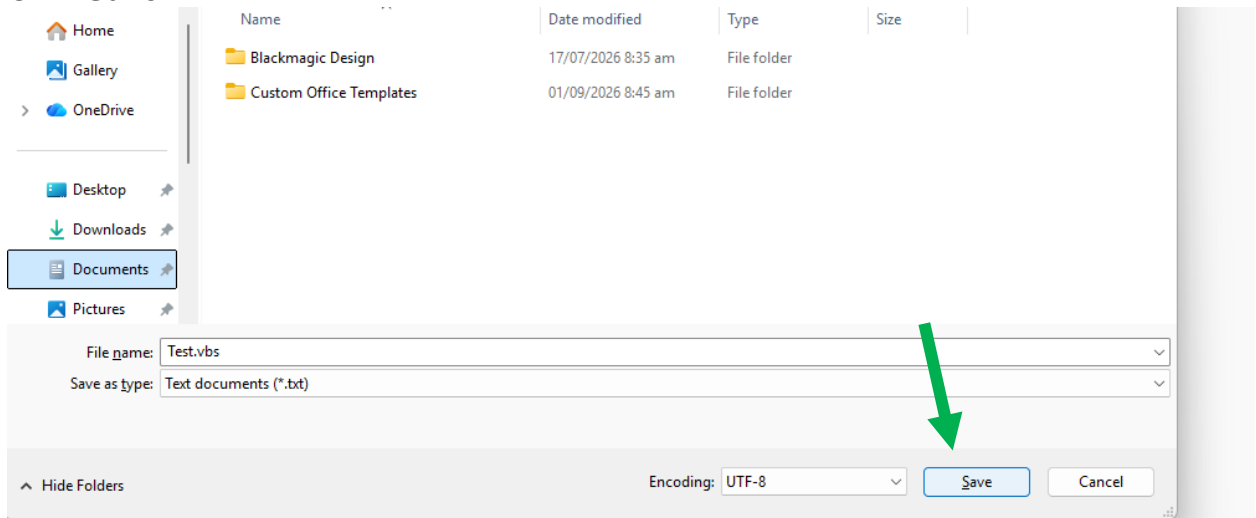


6. In the navigation pane, click **Documents**.

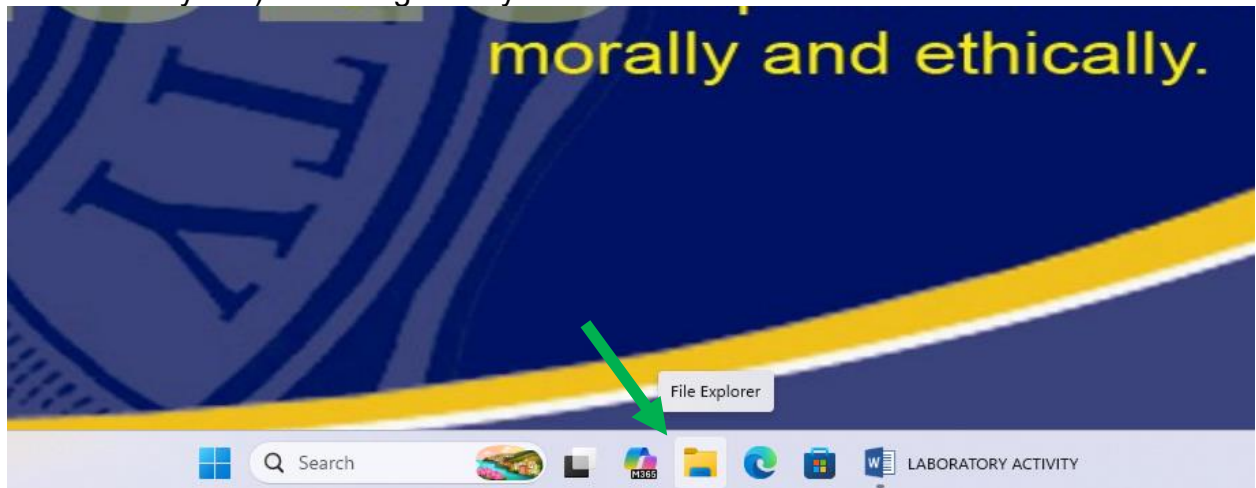




7. Click **Save**.

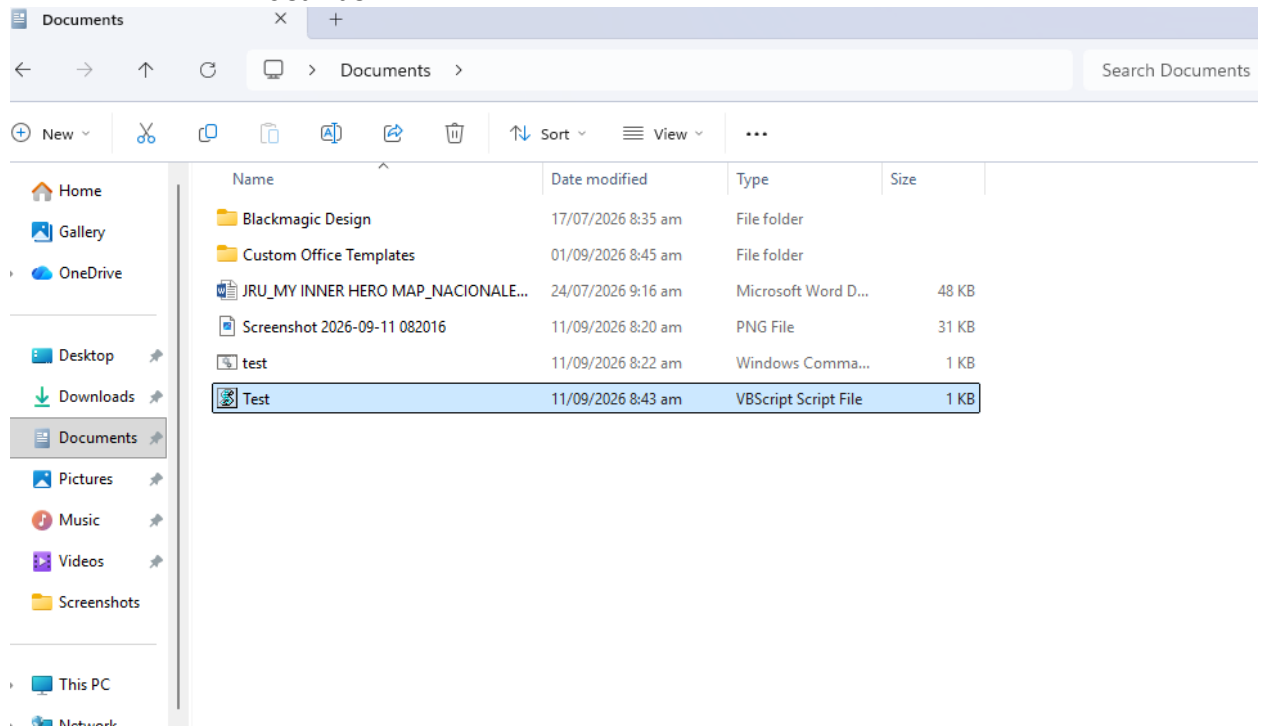


8. Open **File Explorer** (by clicking the folder icon on the taskbar or pressing Windows key + E) and navigate to your Documents folder.



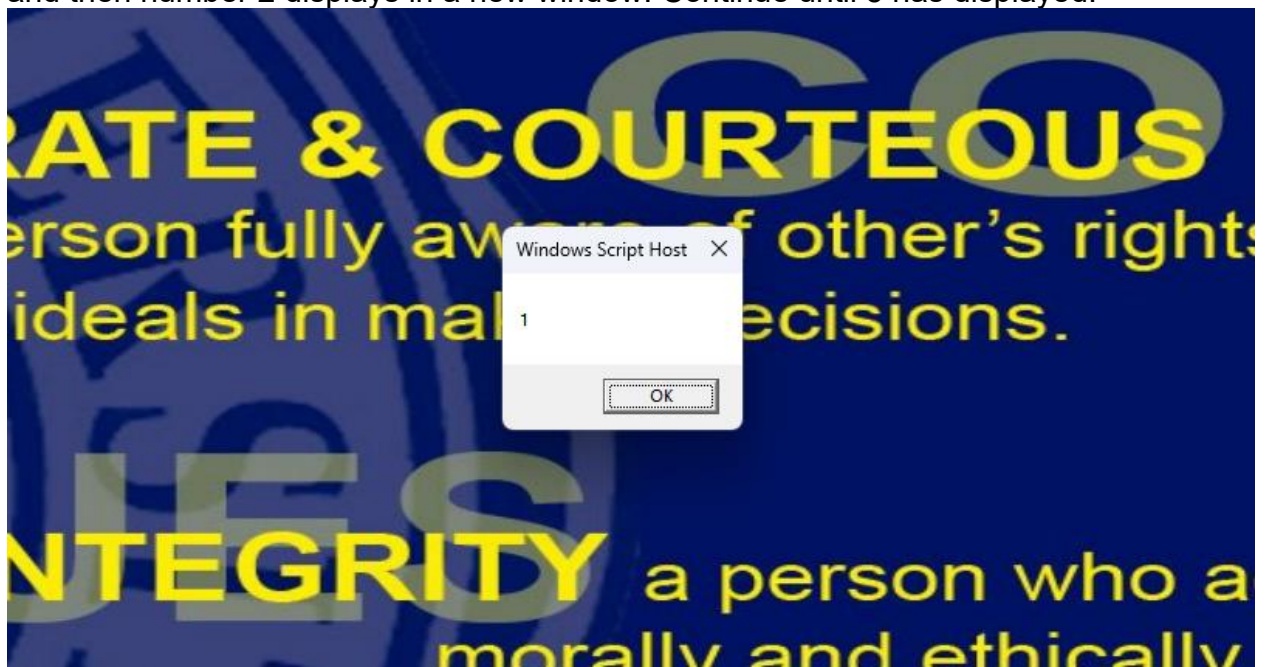


9. Double-click the *Test.vbs* file.



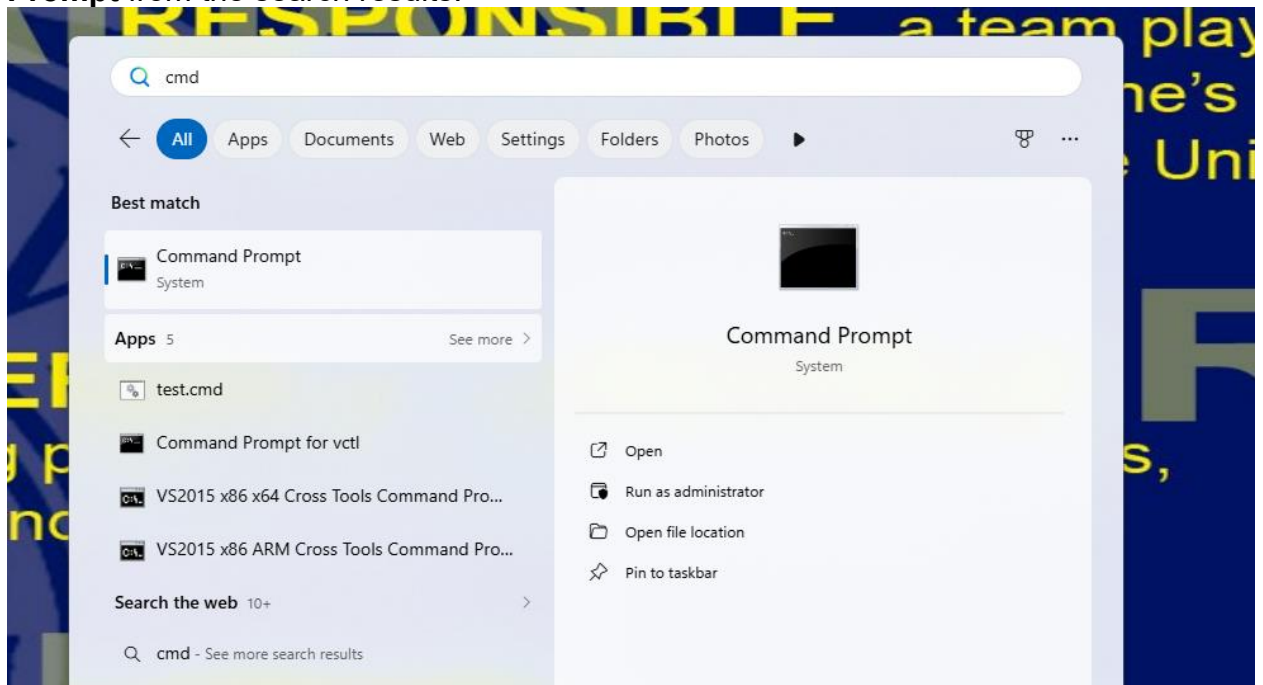


10. The script runs and a new window opens that displays the number 1. Click **OK**, and then number 2 displays in a new window. Continue until 5 has displayed.





11. Click the **Start** button, type cmd in the search bar, and then click **Command Prompt** from the search results.





12. In the Command Prompt window, type *cscript* documents\test.vbs and then press ENTER.

```
Command Prompt
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.

C:\Users\27-200389>cscript documents\Test.vbs|
```

13. The script runs from the command line and displays each number in sequence.

```
Command Prompt
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.

C:\Users\27-200389>cscript documents\Test.vbs
Microsoft (R) Windows Script Host Version 10.0
Copyright (C) Microsoft Corporation. All rights reserved.

1
2
3
4
5

C:\Users\27-200389>|
```



14. Switch to the Notepad window and update the script to read as shown below:

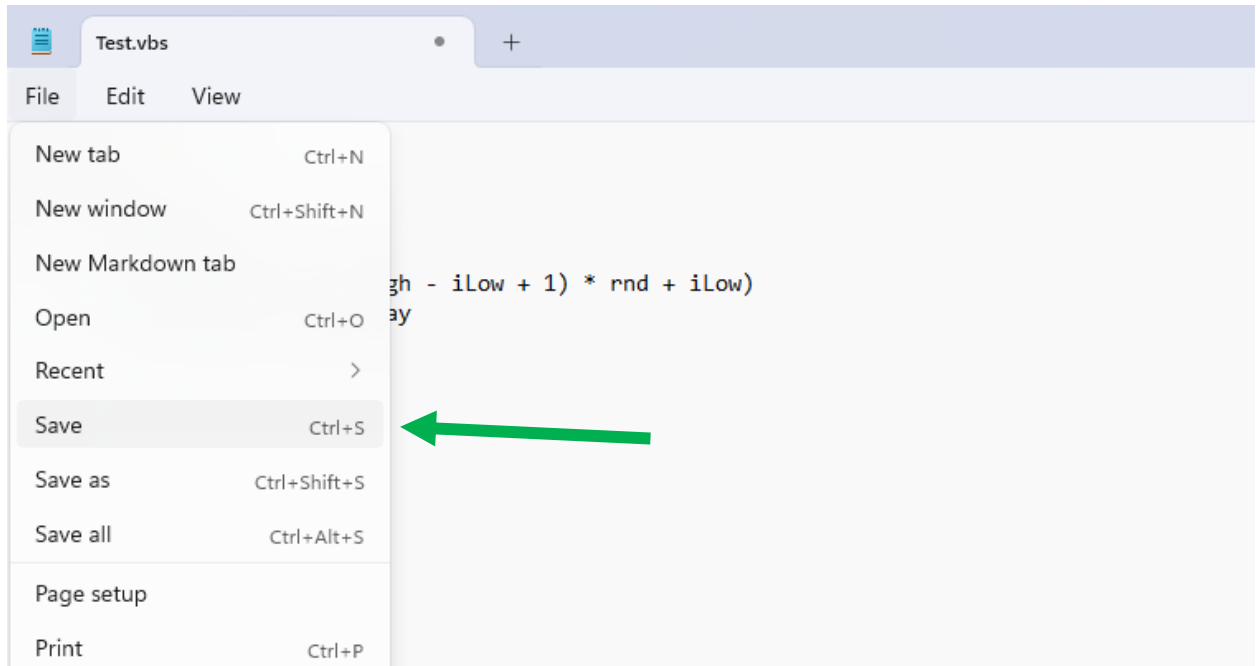
```
iHigh=50  
iLow=10  
for i=1 to 5  
    randomize  
    iDisplay=int((iHigh-iLow+1)*rnd+iLow)  
    wscript.echo iDisplay  
next
```

- This script uses some additional integer variables and built-in VBScript functions to return a random number between 10 and 50 during each iteration of the loop.

```
Test.vbs  
File Edit View  
  
iHigh = 50  
iLow = 10  
for i = 1 to 5  
    randomize  
    iDisplay = int((iHigh - iLow + 1) * rnd + iLow)  
    wscript.echo iDisplay  
next
```

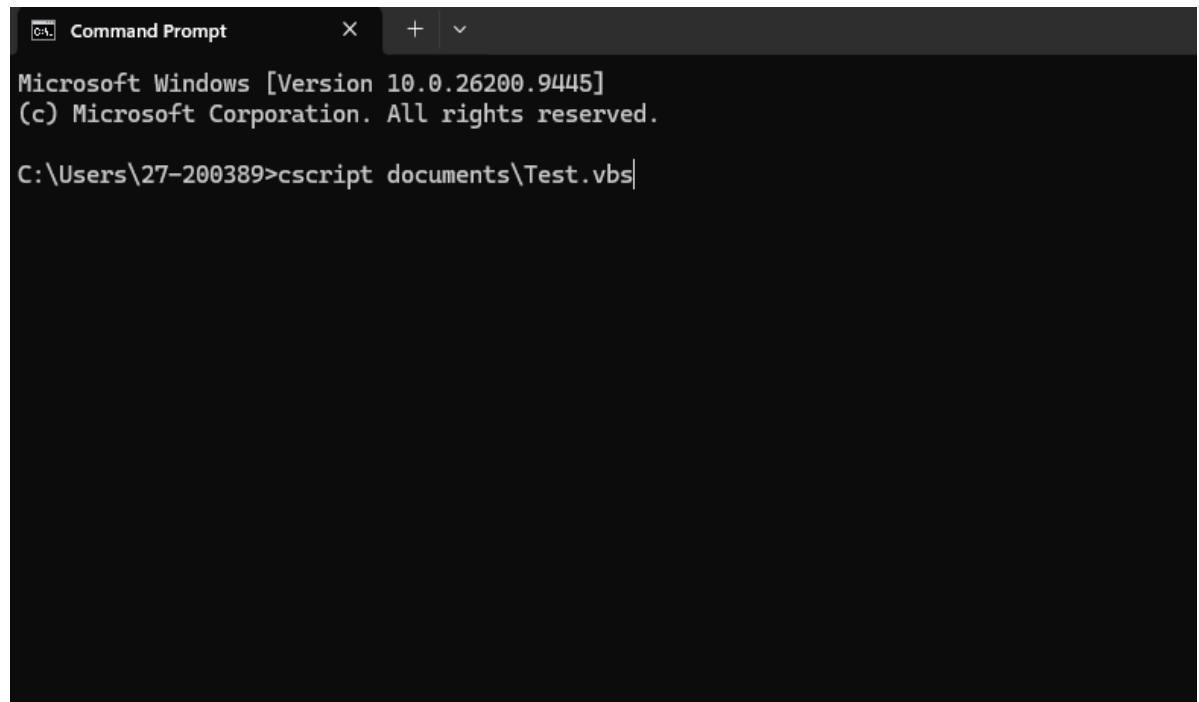


15. Save the file.



16. In the Command Prompt window, type `cscript documents\test.vbs` and then press ENTER.

- Five random numbers between 10 and 50 display.





```
Command Prompt
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.

C:\Users\27-200389>cscript documents\Test.vbs
Microsoft (R) Windows Script Host Version 10.0
Copyright (C) Microsoft Corporation. All rights reserved.

21
18
49
25
17

C:\Users\27-200389>
```

17. What part of the code would you change to show 10 random numbers?

ANSWER: **for i = 1 to 5** change into **for i = 1 to 10**

18. Edit this line and save the file:

```
for i=1 to 10
```

```
Test.vbs
File Edit View

iHigh = 50
iLow = 10
for i = 1 to 10|
    randomize
    iDisplay = int((iHigh - iLow + 1) * rnd + iLow)
    wscript.echo iDisplay
next
```



19. Run the script again.

```
Command Prompt
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.

C:\Users\27-200389>cscript documents\Test.vbs
Microsoft (R) Windows Script Host Version 10.0
Copyright (C) Microsoft Corporation. All rights reserved.

13
10
42
17
50
43
13
21
48
48

C:\Users\27-200389>|
```

20. Switch to the Notepad window and update the script to read as shown below:

```
iHigh=50
iLow=10
iCount=0
sTitle = "Number count"
for i=1 to 10
    randomize
    iDisplay=int((iHigh-iLow+1)*rnd+iLow)
    wscript.echo iDisplay
    if iDisplay > 25 then
        iCount=iCount+1
    end if
next
sMsg = cstr(iCount) + " numbers are greater than 25"
msgbox sMsg, vbOK,sTitle
```

- These modifications add another branch to increment a variable if another variable exceeds a certain value. Note how the branching statements are nested. We also use some string-type variables and a function to convert an integer (*iCount*) into a string for use as text output in a message box. Finally, note the use of *vbOK* in the last line. This is an example of a



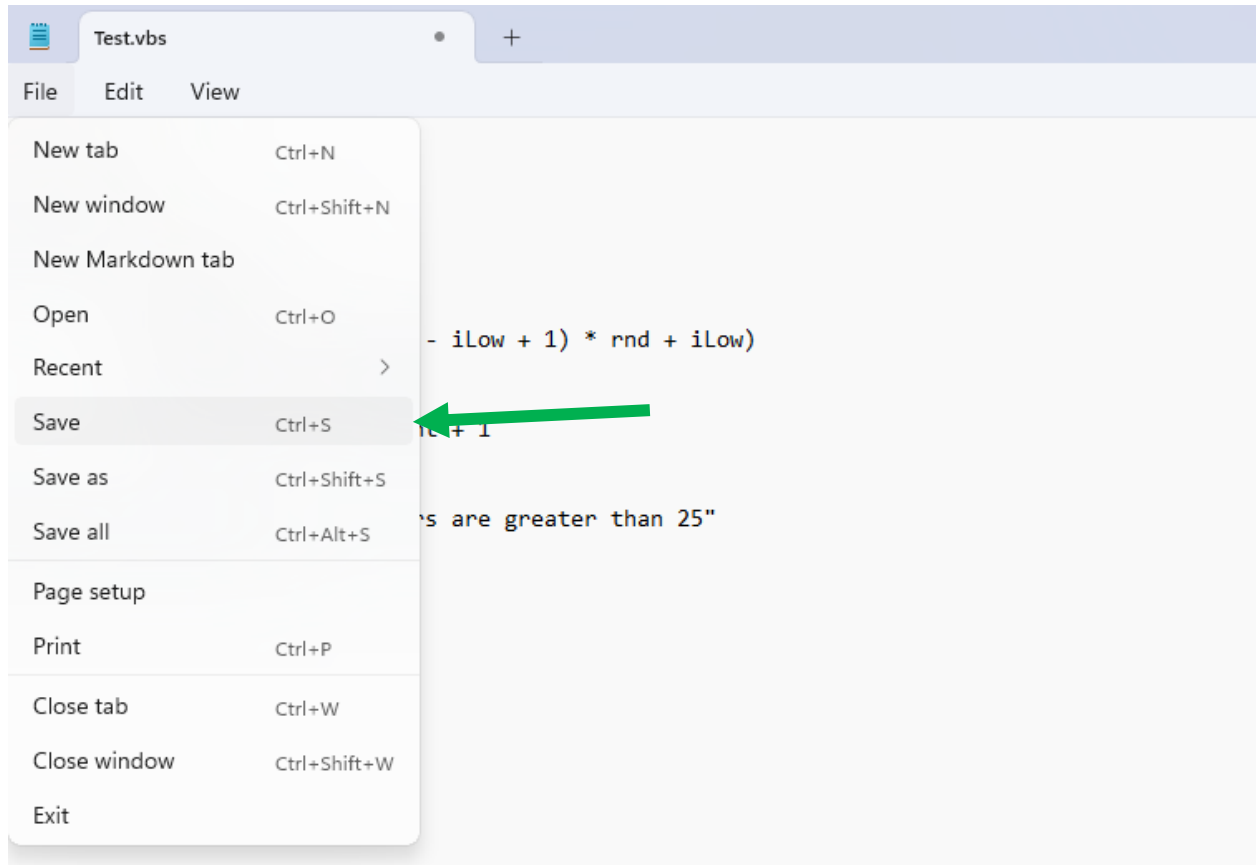
constant. It defines the type of message box to display (one with just an OK button).

```
Test.vbs
File Edit View

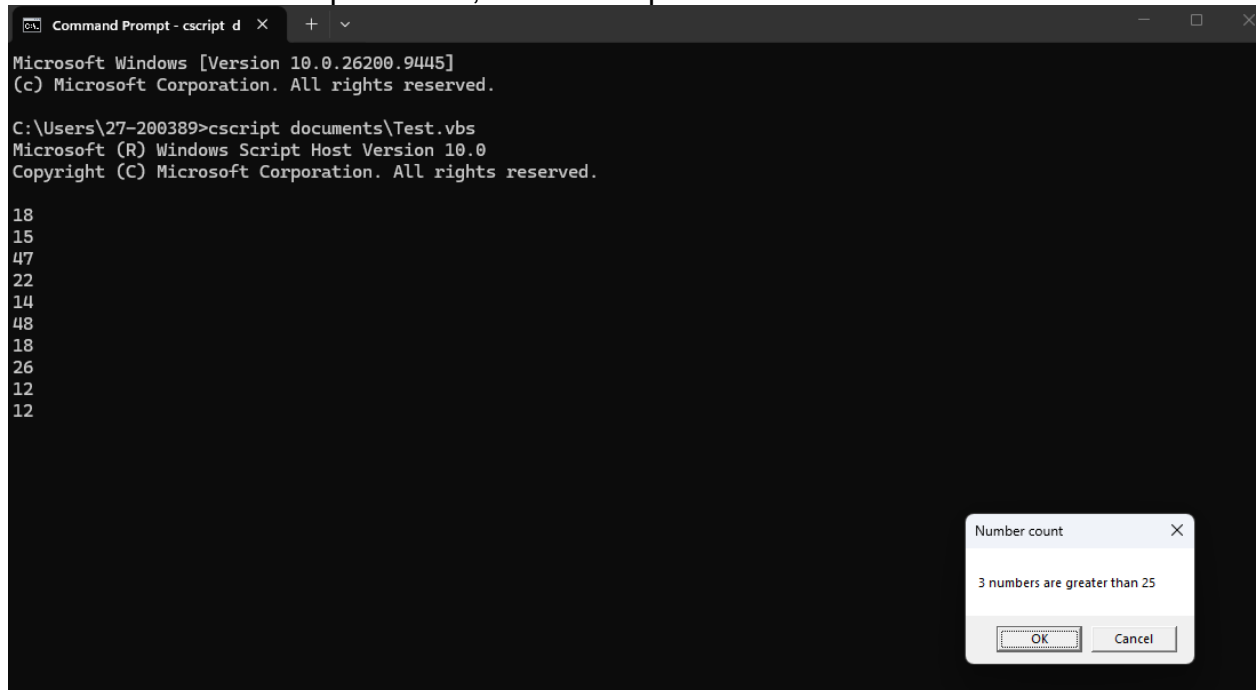
iHigh = 50
iLow = 10
iCount = 0
sTitle = "Number count"
for i = 1 to 10
    randomize
    iDisplay = int((iHigh - iLow + 1) * rnd + iLow)
    wscript.echo iDisplay
    if iDisplay > 25 then
        iCount = iCount + 1
    end if
next
sMsg = cstr(iCount) + " numbers are greater than 25"
msgbox sMsg, vbOK,sTitle
```



21. Save the file.



22. In the Command Prompt window, run the script.



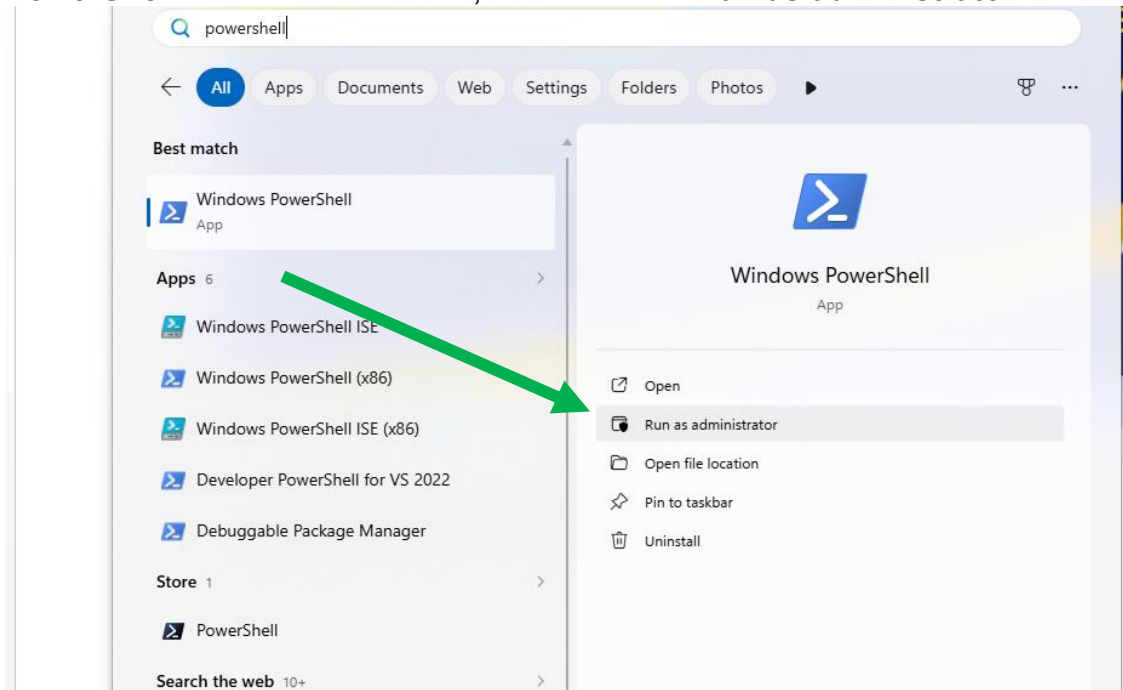


23. Close all open apps and windows.



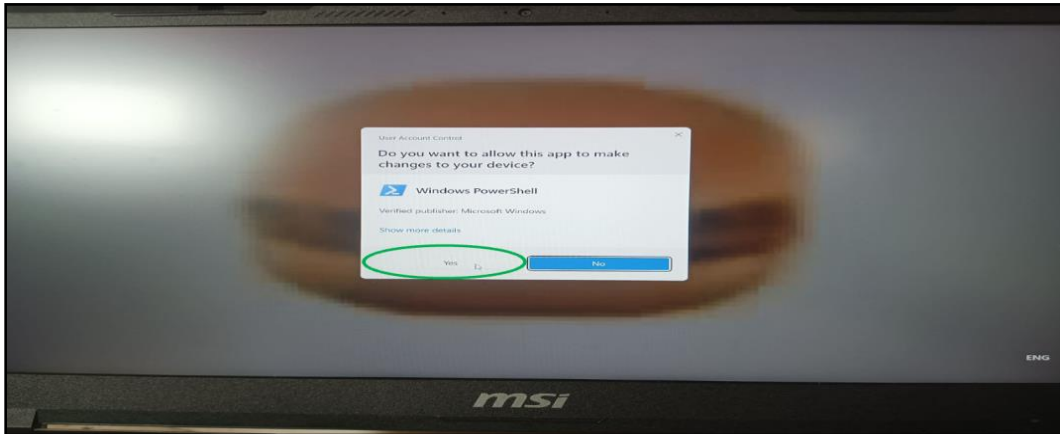
Part 3: Creating a Windows PowerShell Script

1. Click the **Start button**, type Windows PowerShell, right-click **Windows PowerShell** in the search results, and then click **Run as administrator**.





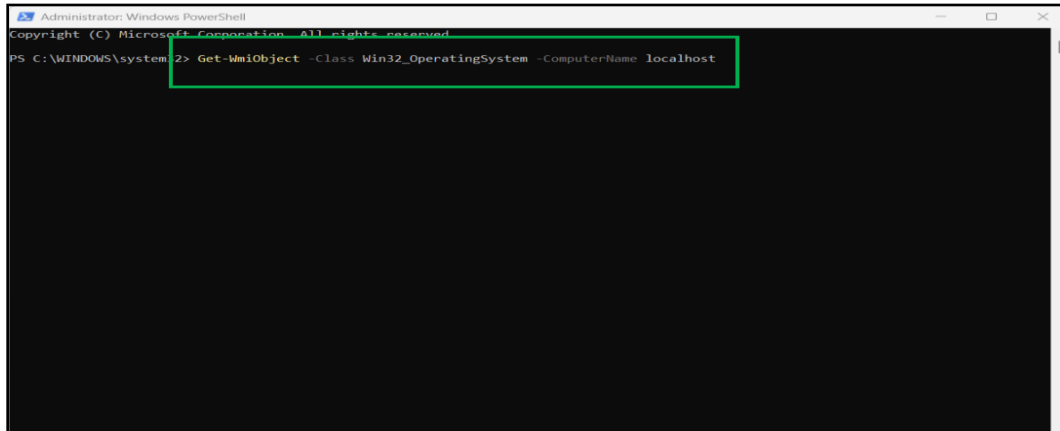
2. In the User Account Control window, click **Yes**.



3. In the Windows PowerShell window, type the following command (ignore the line break) and then press

ENTER:

```
Get-WmiObject -Class Win32_OperatingSystem -ComputerName localhost
```





4. Examine the returned results.

```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32> Get-WmiObject -Class Win32_OperatingSystem -ComputerName localhost

SystemDirectory : C:\WINDOWS\system32
Organization     : 
BuildNumber     : 26200
RegisteredUser   : ssorioktov2100@gmail.com
SerialNumber     : 00356-24774-90685-AAOEM
Version         : 10.0.26200

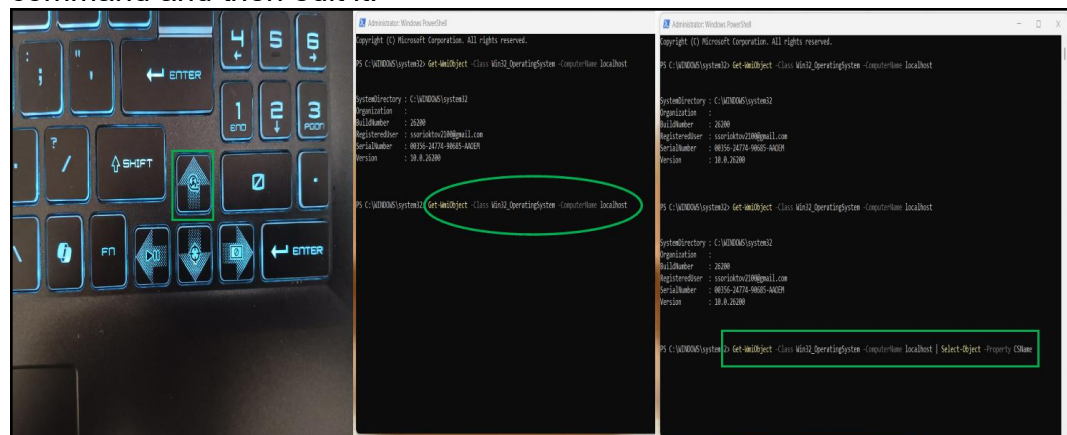
PS C:\WINDOWS\system32>
```

5. In the Windows PowerShell window, type the following command and then press

ENTER:

```
Get-WmiObject -Class Win32_OperatingSystem -ComputerName localhost |
Select-Object -Property CSName
```

- To save a bit of time, you can press the UP ARROW to select the original command and then edit it.



6. Examine the returned results.



- Your computer name is displayed in a table. The command uses a pipe (|) to send the output from one command to another command.

```
Administrator: Windows PowerShell
PS C:\WINDOWS\system32> Get-WmiObject -Class Win32_OperatingSystem -ComputerName localhost | Select-Object -Property CSName
CSName
-----
MSI
PS C:\WINDOWS\system32>
```

7. In the Windows PowerShell window, type the following command and then press ENTER:

Get-Service

```
Administrator: Windows PowerShell
PS C:\WINDOWS\system32> Get-WmiObject -Class Win32_OperatingSystem -ComputerName localhost | Select-Object -Property CSName
CSName
-----
MSI
PS C:\WINDOWS\system32> Get-Service
```

8. Examine the returned results. A list of all services on your computer is displayed.

```
Administrator: Windows PowerShell
PS C:\WINDOWS\system32> Get-Service
Status Name DisplayName
-----
Stopped AarSvc_3b52ae Agent Activation Runtime 3b52ae
Running AdobeUpdateService AdobeUpdateService
Stopped ADPSvc Aggregated Data Platform Service
Stopped ALG Application Layer Gateway Service
Stopped AppIDSvc Application Identity
Running Appinfo Application Information
Stopped AppReadiness App Readiness
Running AppXSvc AppX Deployment Service (AppXSVC)
Stopped ApxSvc Windows Virtual Audio Device Proxy ...
Running AudioEndpointBu... Windows Audio Endpoint Builder
Running Audiosrv Windows Audio
Stopped autotimesvc Cellular Time
Stopped AxInstSV ActiveX Installer (AxInstSV)
Stopped BcastDVRUserSer... GameDVR and Broadcast User Service...
Stopped BDESVC BitLocker Drive Encryption Service
Running BFE Base Filtering Engine
Stopped BITS Background Intelligent Transfer Ser...
Stopped BluetoothUserSe... Bluetooth User Support Service_3b52ae
Running BrokerInfrastru... Background Tasks Infrastructure Ser...
Stopped BTAGService Bluetooth Audio Gateway Service
Stopped BthAvctpSvc AVCTP service
Running bthserv Bluetooth Support Service
Running camsvc Capability Access Manager Service
Stopped CaptureService... CaptureService_3b52ae
Running cbdhsvc_3b52ae Clipboard User Service_3b52ae
Running CDPSSvc Connected Devices Platform Service
Running CDPUserSvc_3b52ae Connected Devices Platform User Ser...
Running CertPropSvc Certificate Propagation
```



9. In the Windows PowerShell window, type the following command and then press ENTER:

```
Get-Service | Select DisplayName
```

A screenshot of a Windows PowerShell window titled "Administrator: Windows PowerShell". The window shows the command prompt with the command `Get-Service | Select DisplayName` entered. The command is highlighted with a green box. The output of the command is not yet visible.

10. Examine the returned results. The same list of services is displayed but this time the output is filtered down to the service's "friendly" name.

A screenshot of a Windows PowerShell window titled "Administrator: Windows PowerShell". The window shows the command prompt with the command `Get-Service | Select DisplayName` entered. The command is highlighted with a green box. The output of the command is displayed as a list of service names, with the first few lines highlighted by a green box. The output is as follows:
DisplayName

Agent Activation Runtime_3b52ae
AdobeUpdateService
Aggregated Data Platform Service
Application Layer Gateway Service
Application Identity
Application Information
App Readiness
AppX Deployment Service (AppXSVC)
Windows Virtual Audio Device Proxy Service
Windows Audio Endpoint Builder
Windows Audio
Cellular Time
ActiveX Installer (AxInstSV)
GameDVR and Broadcast User Service_3b52ae
BitLocker Drive Encryption Service
Base Filtering Engine
Background Intelligent Transfer Service
Bluetooth User Support Service_3b52ae
Background Tasks Infrastructure Service
Bluetooth Audio Gateway Service
AVCTP service
Bluetooth Support Service
Capability Access Manager Service

11. In the Windows PowerShell window, type the following command and then press ENTER:



```
Get-WmiObject win32_service | Select DisplayName, State | Sort State, DisplayName
```

```
Administrator: Windows PowerShell
Security Center
Windows Search
Windows Update
Microsoft Usage and Quality Insights
WMAN AutoConfig
Xbox Live Auth Manager
Xbox Live Game Save
Xbox Accessory Management Service
Xbox Live Networking Service

PS C:\WINDOWS\system32> Get-WmiObject win32_service | Select DisplayName, State | Sort State, DisplayName
```

12. Examine the returned results. A list of all services on your computer is displayed, sorted by state and then display name.

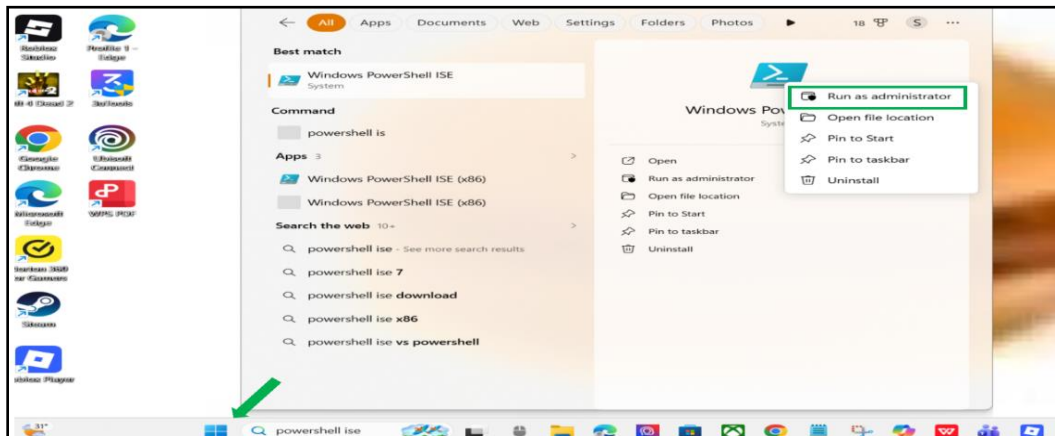
- So far we have been running cmdlets individually. You can also create PowerShell scripts in the editor environment and save them as files.

```
Administrator: Windows PowerShell
PS C:\WINDOWS\system32> Get-WmiObject win32_service | Select DisplayName, State | Sort State, DisplayName

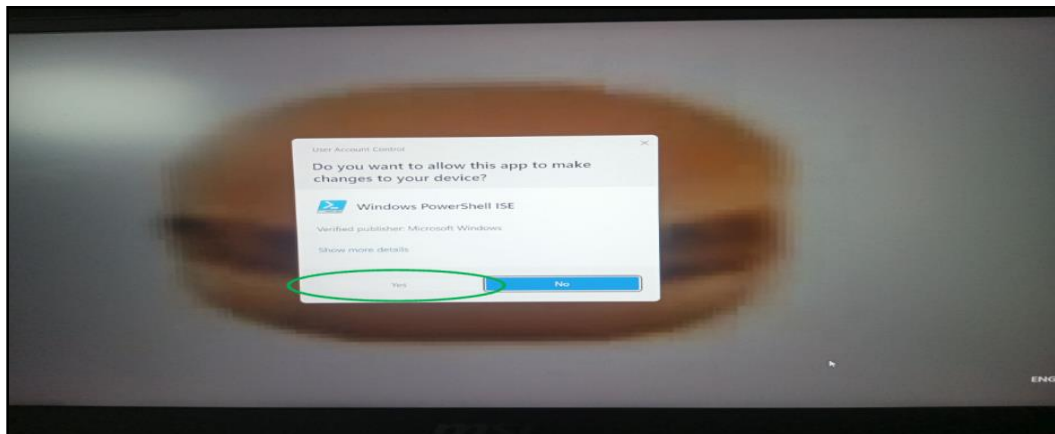
DisplayName                                     State
-----
AdobeUpdateService                             Running
Application Information                         Running
Background Tasks Infrastructure Service          Running
Base Filtering Engine                           Running
Bluetooth Support Service                       Running
Capability Access Manager Service               Running
Certificate Propagation                         Running
Clipboard User Service_3b52ae                   Running
CMG Key Isolation                              Running
COM+ Event System                              Running
Connected Devices Platform Service              Running
Connected Devices Platform User Service_3b52ae Running
Connected User Experiences and Telemetry         Running
Contact Data_3b52ae                             Running
CoreMessaging                                  Running
Credential Manager                             Running
Cryptographic Services                         Running
Data Usage                                      Running
DCOM Server Process Launcher                    Running
Device Association Service                      Running
DevicesFlow_3b52ae                             Running
DevQuery Background Discovery Broker             Running
DHCP Client                                     Running
Diagnostic Policy Service                       Running
Diagnostic System Host                          Running
```



13. Click the **Start** button, type Windows PowerShell ISE, right-click **Windows PowerShell ISE** in the search results, and then click **Run as administrator**.



14. In the User Account Control window, click **Yes**.



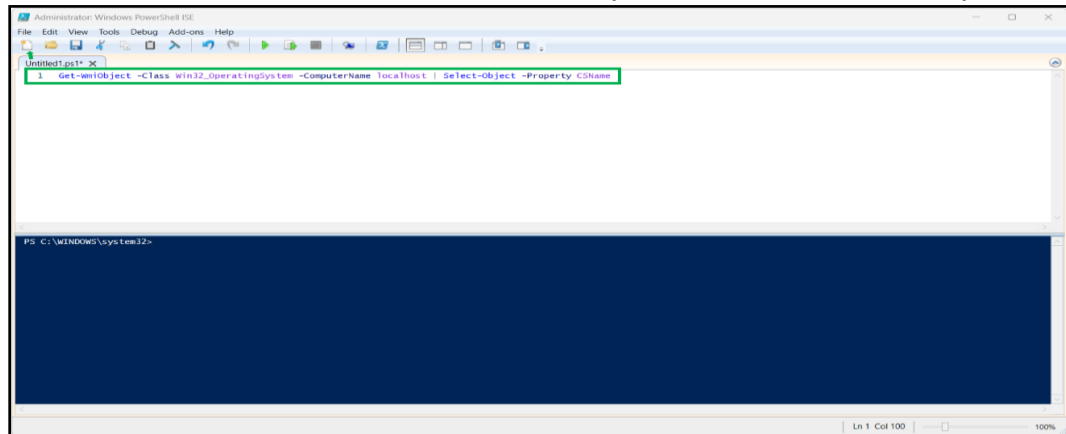
15. In the Untitled1.ps1 area, type the following code as a single line:

```
Get-WmiObject -Class Win32_OperatingSystem -ComputerName localhost |  
Select-Object -Property CSName
```

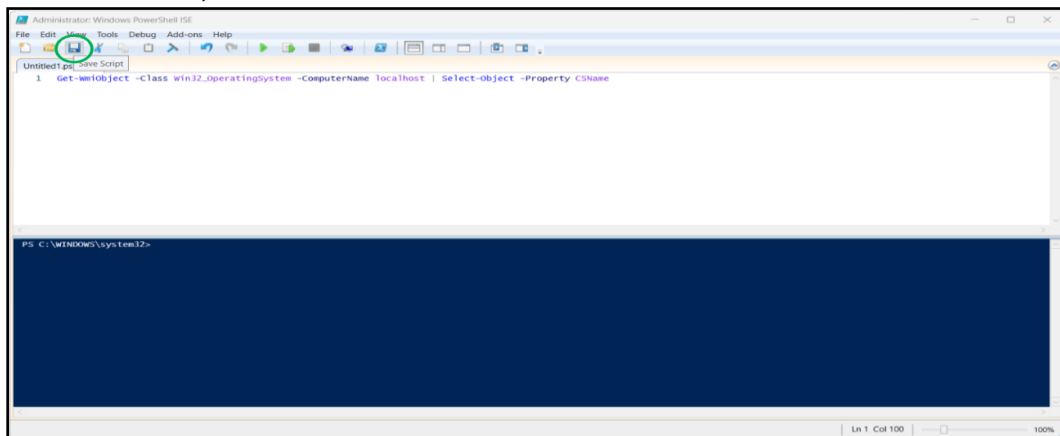
- Note that as you type, suggestion boxes appear helping you to autocomplete statements. Also notice that the editor applies color-coding to distinguish parts of your code. These features make a development



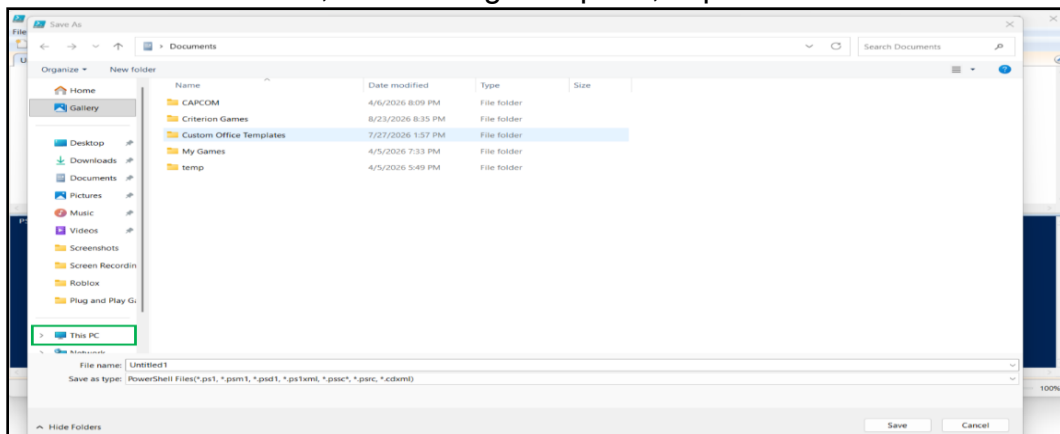
environment much easier to work in than a plain text editor like Notepad.



16. On the toolbar, click **Save**.

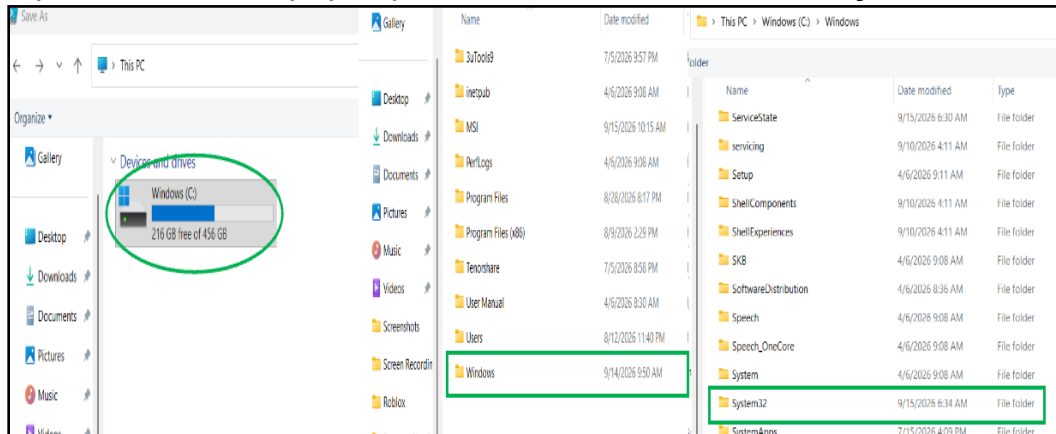


17. In the Save As window, in the navigation pane, expand **This PC**.

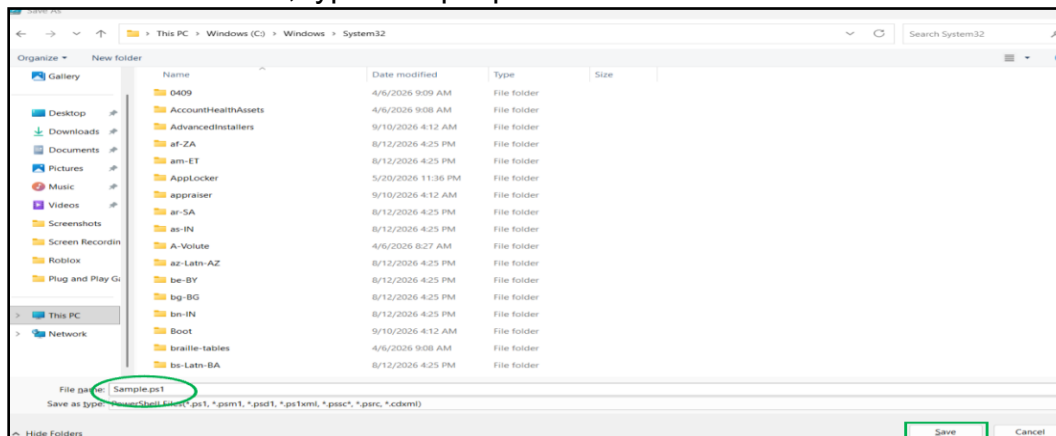




18. Expand **Local Disk (C:)**, expand **Windows**, and then click **System32**.

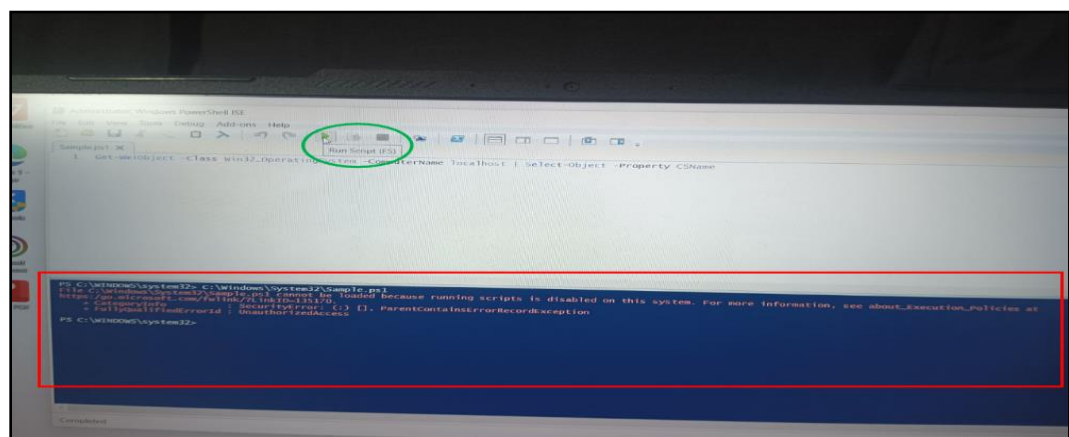


19. In the File name box, type **Sample.ps1** and then click **Save**.



20. In the Administrator: Windows PowerShell ISE window, on the toolbar, click **Run**.

- You receive an error. This is because unsigned scripts are blocked by default. To enable scripts without a valid coding certificate to run, switch to the Windows PowerShell window.



21. In the Administrator: Windows PowerShell window, type the following command and then press ENTER:



Set-ExecutionPolicy RemoteSigned

```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32> Set-ExecutionPolicy RemoteSigned

Execution Policy Change
The execution policy helps protect you from scripts that you do not trust. Changing the execution policy might expose
you to the security risks described in the about_Execution_Policies help topic at
https://go.microsoft.com/fwlink/?LinkID=135170. Do you want to change the execution policy?
[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "N"):
```

22. When prompted, type A and press ENTER.

```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32> Set-ExecutionPolicy RemoteSigned

Execution Policy Change
The execution policy helps protect you from scripts that you do not trust. Changing the execution policy might expose
you to the security risks described in the about_Execution_Policies help topic at
https://go.microsoft.com/fwlink/?LinkID=135170. Do you want to change the execution policy?
[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "N"): A
```

23. Switch to Windows PowerShell ISE and click **Run**. Your simple one-line script runs.

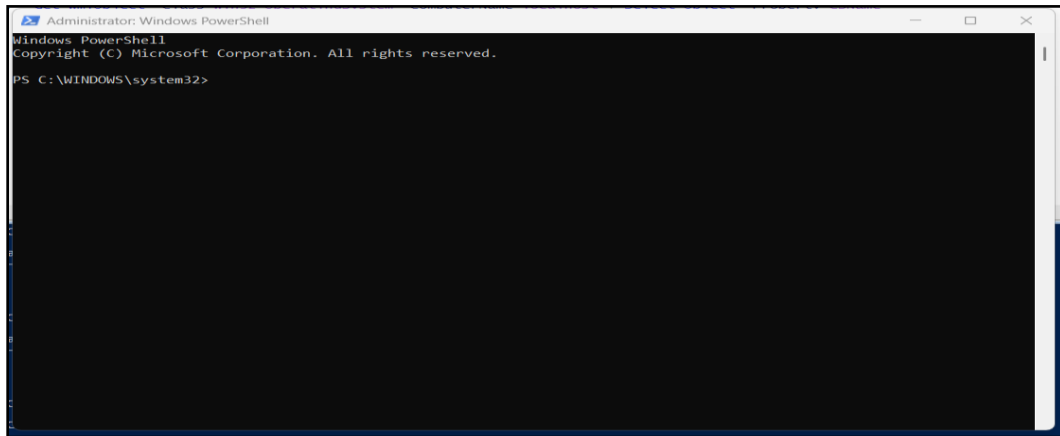
```
Administrator: Windows PowerShell ISE
File Edit View Tools Debug Add-ons Help
Sample.ps1 x Run Script (R)
1 Get-WmiObject -Class Win32_OperatingSystem -ComputerName localhost | Select-Object -Property CSName

PS C:\WINDOWS\system32> C:\Windows\System32\Sample.ps1
CSName
-----
MSI

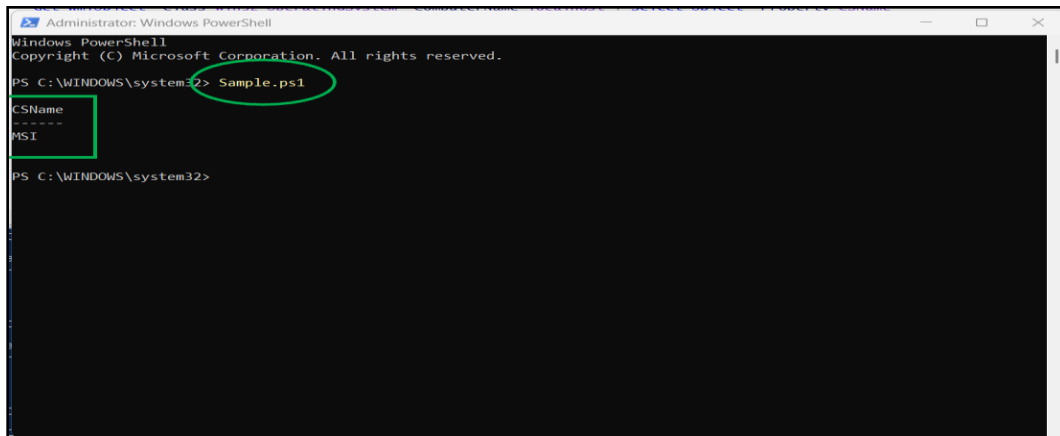
PS C:\WINDOWS\system32> C:\Windows\System32\Sample.ps1
CSName
-----
MSI
```



24. Switch to the Windows PowerShell window.



25. At the prompt, type Sample.ps1 and then press Enter. Your simple one-line script runs.



26. Close all open windows and apps.





V. REFLECTION:

Upon successful completion of "Lab 2.3.8: Using Scripting Engine," write a comprehensive reflection (between 150-250 words) addressing the following points:

- **Learning Outcomes:** What new knowledge or skills did you acquire or reinforce regarding the use of a scripting engine?
- **Challenges and Solutions:** Describe any difficulties or unexpected issues you encountered during the lab. How did you overcome them, or what steps would you take to resolve them in the future?
- **Real-world Relevance:** How do the skills and understanding gained in this lab relate to real-world scenarios, either in your personal life or a professional IT context?
- **Critical Thinking:** If you had to use an internet image for any step, explain why you couldn't perform the action physically and how finding the image contributed to your understanding.

Completing this lab activity, gave me a clearer understanding of how batch files, VBScript, and PowerShell work together to automate simple system and network tasks. We learned how to write a basic batch file to generate a network configuration report, create loops and conditional statements in VBScript, and use PowerShell cmdlets to retrieve and filter system information. Reinforcing concepts like variables, conditionals, and execution policies helped us see how different scripting tools approach similar programming logic in different ways.

The main challenge we faced was being unfamiliar with the scripting tools themselves, especially the syntax differences between VBScript and PowerShell. To resolve this, we researched each tool online, looking into tutorials and documentation to understand the correct commands and structure needed to complete the activity successfully.

In terms of real-world relevance, these skills are valuable in IT work, since automating repetitive tasks like generating reports or checking system information saves time and reduces error, especially for system administrators managing multiple machines.

For the critical thinking portion, we did not need to use any internet image for this activity, as every step could be performed directly through the command prompt and PowerShell ISE on our own device.